Statistical Visualizations with Seaborn & Matplotlib

Lesson Document & Learning Materials | Beginner to Intermediate

1. Lesson Overview

Data visualization is the process of presenting information through charts and graphs. In data science, visualizations help us understand patterns, compare values, identify unusual observations, and communicate findings. Matplotlib and Seaborn are two popular Python libraries used to create visualizations.

2. Learning Objectives

- Understand why statistical visualization is important.
- Create basic charts using Matplotlib.
- Build statistical plots using Seaborn.
- Choose suitable charts for different types of data.
- Interpret distributions, relationships, and outliers.

3. Introduction to Matplotlib

Matplotlib is a Python plotting library. Its pyplot module provides simple functions for creating line charts, bar charts, histograms, scatter plots, and many other visualizations.

Install and import

```
pip install matplotlib seaborn pandas

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

4. Common Matplotlib Charts

Line chart – shows changes or trends over an ordered sequence.

```
months = ['Jan', 'Feb', 'Mar', 'Apr']
sales = [120, 150, 135, 180]

plt.plot(months, sales, marker='o')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.show()
```

Bar chart – compares values across categories.

```
subjects = ['Maths', 'Science', 'English']
marks = [85, 90, 78]

plt.bar(subjects, marks)
plt.title('Subject Marks')
plt.xlabel('Subject')
plt.ylabel('Marks')
plt.show()
```

Histogram – displays the frequency distribution of numerical data.

```
marks = [45, 55, 60, 62, 70, 72, 75, 80, 85, 90]

plt.hist(marks, bins=5, edgecolor='black')
plt.title('Marks Distribution')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.show()
```

Scatter plot – shows the relationship between two numerical variables.

```
hours = [1, 2, 3, 4, 5, 6]
marks = [40, 48, 55, 65, 72, 85]

plt.scatter(hours, marks)
plt.title('Study Hours vs Marks')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.show()
```

5. Introduction to Seaborn

Seaborn is a statistical data visualization library built on top of Matplotlib. It works especially well with Pandas DataFrames and provides attractive statistical charts with less code.

Sample dataset

```
import seaborn as sns
import matplotlib.pyplot as plt

# Built-in example dataset
tips = sns.load_dataset('tips')
print(tips.head())
```

6. Important Seaborn Statistical Plots

Count plot – counts how many records belong to each category.

```
sns.countplot(data=tips, x='day')
plt.title('Number of Bills by Day')
plt.show()
```

Box plot – summarizes a numerical distribution and helps identify possible outliers.

```
sns.boxplot(data=tips, x='day', y='total_bill')
plt.title('Bill Distribution by Day')
plt.show()
```

Violin plot – combines a box-plot summary with a view of distribution density.

```
sns.violinplot(data=tips, x='day', y='total_bill')
plt.title('Bill Distribution by Day')
plt.show()
```

Histogram with KDE – shows frequencies and a smoothed estimate of the distribution.

```
sns.histplot(data=tips, x='total_bill', kde=True)
plt.title('Total Bill Distribution')
plt.show()
```

Heatmap – uses color intensity to show values in a matrix, often a correlation matrix.

```
numeric_tips = tips.select_dtypes(include='number')
corr = numeric_tips.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

7. Which Chart Should You Use?

Purpose	Recommended chart
Compare categories	Bar chart / Count plot
Show a trend over time	Line chart
Understand a distribution	Histogram / KDE plot
Compare distributions and spot outliers	Box plot / Violin plot
Explore relationships between two numeric variables	Scatter plot
Display a matrix of values or correlations	Heatmap

8. Mini Project: Student Marks Analysis

Goal: visualize student marks and investigate how study hours relate to marks. The following example uses a small sample dataset.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    'student': ['A', 'B', 'C', 'D', 'E', 'F'],
    'study_hours': [1, 2, 3, 4, 5, 6],
    'marks': [42, 50, 58, 67, 76, 88]
}
df = pd.DataFrame(data)

sns.scatterplot(data=df, x='study_hours', y='marks')
plt.title('Study Hours vs Marks')
plt.show()

sns.histplot(data=df, x='marks', bins=5, kde=True)
plt.title('Distribution of Student Marks')
plt.show()
```

Interpretation: the scatter plot helps inspect whether marks tend to rise with study hours. The histogram shows how the marks are distributed. This small example illustrates visualization; it is not enough to prove that study hours cause higher marks.

9. Good Visualization Practices

- Give every chart a clear, meaningful title.
- Label axes and include units where needed.
- Choose a chart that matches the question and data type.
- Avoid unnecessary decorations and misleading scales.
- Use readable labels and explain important findings.
- Check missing values and unusual observations before interpreting results.

10. Practice Activities

1. Create a bar chart for marks in five subjects.
2. Create a histogram for a list of exam scores.
3. Use a scatter plot to compare study hours and marks.
4. Create a box plot to compare total bills across days in the tips dataset.
5. Create a heatmap from the numeric columns of a dataset and describe one observed relationship.

11. Learning Outcomes

After this lesson, learners should be able to create common charts with Matplotlib, make statistical visualizations with Seaborn, select appropriate plots, and explain basic patterns and distributions shown in charts.

Prerequisites: Basic Python, lists, functions, and introductory Pandas knowledge.
Suggested tools: Jupyter Notebook, Google Colab, or VS Code.
Suggested duration: 2–3 study sessions.